

CodroidCPP SDK Manual

Version: 2.1.2 | Namespace: `Codroid`

Table of Contents

#	Section	Description
1	Quick Start	Build, connect, and run your first program
2	Core Concepts	Lifecycle, TCP model, units, exceptions
3	CodroidClient API	Complete CodroidClient API reference
4	Motion API	JointPoint, CartesianPoint, MoveInstruction, MotionWaitOptions, enums
5	Data Types and Enums	CommandResult, ClientRealtimeState, RobotFrame, exceptions
6	CRI Real-Time Data and Control	CriRealtimeDispatcher, TrajectoryGenerator, TrajectoryRequest
7	IO and Register	DI/DO/AI/AO operations, register read/write
8	Utilities	Publish/Subscribe, GlobalVariables, Kinematics, ConsoleUtf8

Environment Requirements

Platform	Compiler	Build Tool
Linux	GCC 9+ / Clang 10+	CMake 3.14+
Windows	MSVC 2019/2022/2026	CMake 3.14+
Windows	MinGW-w64 (MSYS2)	CMake 3.14+

Dependencies

- **Asio** (standalone, not Boost) — Networking
- **nlohmann/json** — JSON serialization
- **GoogleTest** (optional) — Unit testing

Build

Linux

```
chmod +x build_linux.sh
./build_linux.sh
```

Output in `build_linux/` (contains `libCodroid.so` and example programs).

Windows (MSVC)

```
build_msvc.bat
```

Script supports:

- `1`: Visual Studio 2019
- `2`: Visual Studio 2022
- `3`: Visual Studio 2026

Output in `build_msvc/Debug` and `build_msvc/Release`.

Windows (MinGW)

```
build_mingw.bat
```

Output in `build_mingw/`.

API Naming Convention

All public methods use **PascalCase** naming, consistent with C# and Python SDKs.

```
// Correct
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
```

Unit Convention

Layer	Linear	Angular
SDK public API	mm	deg (degrees)
TCP JSON protocol	mm	deg
CRI UDP binary (wire)	m	rad (radians)
<code>ClientRealtimeState</code> (parsed)	mm	deg

`CriRealtimeDispatcher` with `convert_to_si=true` (default) automatically converts mm/deg to m/rad.

Firmware Requirement

All SDK interfaces require controller firmware $\geq$ **2.3.3.43**.

See `Codroid::MinControllerFirmware` in `CodroidDefine.h`.

Quick Start

Build SDK

Linux

```
# Clone repository
git clone <repository-url>
cd CodroidCPP

# Build
chmod +x build_linux.sh
./build_linux.sh
```

Output in `build_linux/`:

- `libCodroid.so` — Dynamic library
- `examples/` — Example programs

Windows (MSVC)

```
REM After cloning
cd CodroidCPP

REM Build
build_msvc.bat
```

Select Visual Studio version:

- `1`: Visual Studio 2019
- `2`: Visual Studio 2022
- `3`: Visual Studio 2026

Output in `build_msvc/Debug` and `build_msvc/Release`.

Windows (MinGW)

```
cd CodroidCPP
build_mingw.bat
```

Output in `build_mingw/`.

Minimal Example

Connect to the controller, read a digital input, write a digital output, and disconnect.

```
#include "Codroid/client.hpp"
```

```

#include <iostream>

int main() {
    Codroid::CodroidClient robot;

    // Connect, enter remote mode, and power on
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "Connection failed" << std::endl;
        return 1;
    }

    // Read DI port 0
    int di0 = robot.GetDi(0);
    std::cout << "DI 0 = " << di0 << std::endl;

    // Write DI value to DO port 10
    robot.SetDo(10, di0);

    // Disconnect
    robot.Disconnect();
    return 0;
}

```

Complete Workflow Example

```

#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"
#include <iostream>

int main() {
    // Windows console UTF-8 support
#ifdef _WIN32
    Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif

    Codroid::CodroidClient robot;

    // 1. Connect
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "Connection failed" << std::endl;
        return 1;
    }

    // 2. IO operations
    int di0 = robot.GetDi(0);
    robot.SetDo(10, di0);

    // 3. Register
    double regValue = robot.GetRegisterValue(49100);
    robot.SetRegisterValue(49100, regValue + 1);
}

```

```

// 4. Motion
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// 5. Blocking motion
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
robot.MovLSync(target, 150, 500);

// 6. Disconnect
robot.Disconnect();
return 0;
}

```

Run Example Programs

```

# Linux
cd build_linux
./examples/01_basic_usage 192.168.8.136

# Windows (MSVC)
cd build_msvc\Release
01_basic_usage.exe 192.168.8.136

```

Error Handling

TCP command behavior depends on `SetThrowOnCommandError` setting:

Default Mode (No Exception)

```

Codroid::CodroidClient robot;
robot.Connect("192.168.8.136");

auto result = robot.SetDo(999, 1); // Invalid port
if (!result.Ok()) {
    std::cerr << "Controller error: " << result.error_msg << std::endl;
}

```

Exception Mode

```
robot.SetThrowOnCommandError(true);

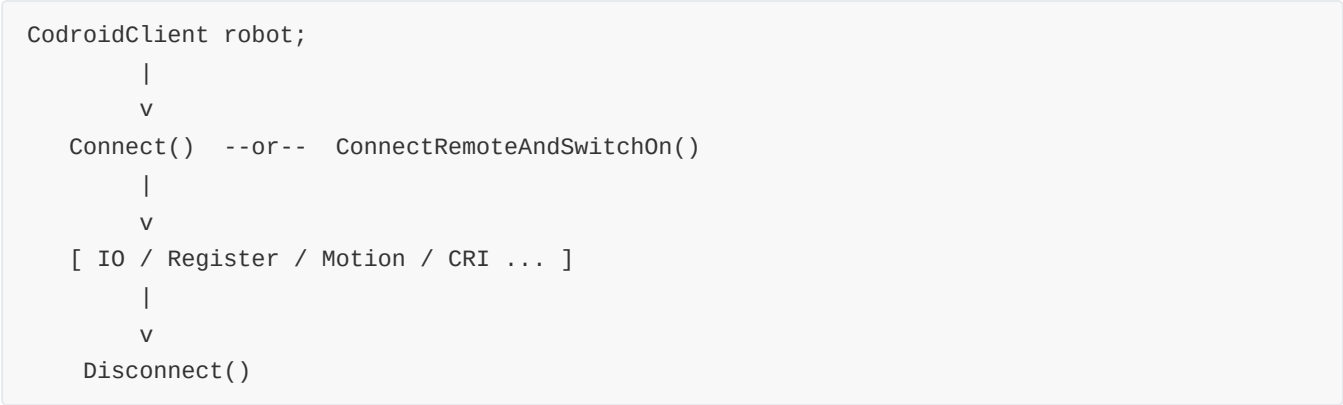
try {
  robot.SetDo(999, 1); // Invalid port
} catch (const Codroid::CodroidCommandException& ex) {
  std::cerr << "Controller error: " << ex.controller_error() << std::endl;
}
```

Exception Types

Exception	Condition
CodroidCommandException	Controller returns err field
CodroidException	General runtime error
std::runtime_error	Standard library exception

Core Concepts

CodroidClient Lifecycle



```
Codroid::CodroidClient robot;

try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... use robot ...
} catch (...) {
    robot.Disconnect(); // Always call
    throw;
}
robot.Disconnect();
```

Constructor

```
Codroid::CodroidClient robot;
```

- Default construction, no connection
- Call `Connect` or `ConnectRemoteAndSwitchOn` to establish connection
- TCP port defaults to **9001**

Properties

Method	Return Type	Description
<code>GetRobotRealtimeState()</code>	<code>ClientRealtimeState</code>	Thread-safe CRI data snapshot
<code>GetCriUdpListenPort()</code>	<code>int</code>	CRI UDP listen port

Callback

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "Joints: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

Fires after each valid CRI UDP frame is parsed. Callback executes on internal receive thread — avoid long blocking.

TCP Command Model

Every SDK method that communicates with the controller follows this pattern:

1. SDK assigns a unique `id`
2. SDK serializes `{ id, ty, db }` as JSON and sends over TCP
3. Controller responds with `{ id, ty, db, err }`
4. SDK matches response by `id`
5. If `err` is non-empty → `CommandResult::error_msg` is non-empty (or throws exception)
6. If no response within 10s → timeout

CommandResult

```
struct CommandResult {
    int id = 0;           // Request id
    std::string ty;       // Response type
    std::string error_msg; // Error message (empty = success)
    std::string raw_json; // Complete response JSON

    bool Ok() const noexcept; // Returns true if error_msg is empty
};
```

Most methods return `CommandResult`. Check `Ok()` for success.

Unit Convention

SDK public APIs use **millimeters** and **degrees**. This matches the TCP JSON protocol.

Context	Linear	Angular
SDK API, TCP JSON	mm	deg
CRI UDP wire format	m	rad
<code>ClientRealtimeState</code> (parsed)	mm	deg

Important: CRI UDP binary payloads use meters and radians. SDK automatically converts to mm/deg internally. Do not assume raw UDP floats are mm/deg.

Naming Convention

All public methods use **PascalCase**, consistent with C# and Python SDKs.

```
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
robot.SetDo(10, 1);
```

Thread Safety

- `GetRobotRealtimeState()` — Thread-safe (returns clone)
- `SetCriDataReceived(callback)` — Thread-safe
- All TCP methods — Safe to call from any thread, but do not call concurrently on the same `CodroidClient`
- `CriRealtimeDispatcher::SendCommand` / `SendTrajectory` — Not thread-safe

Exception Types

Exception	Condition	Source
<code>CodroidCommandException</code>	Controller returns <code>err</code> field	TCP response
<code>CodroidException</code>	General runtime error	SDK internal
<code>std::runtime_error</code>	Standard library exception	Standard library

CodroidCommandException Properties

```
class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;           // Protocol request ID
    const std::string& command_ty() const noexcept; // e.g. "Robot/move"
    const std::string& controller_error() const noexcept; // Controller err field
    const std::string& raw_response_json() const noexcept; // Complete response
};
```

CodroidClient API Reference

Class: `CodroidClient`

Namespace: `Codroid`

Header: `Codroid/client.hpp`

Construction and Destruction

CodroidClient()

Default constructor, no connection.

```
Codroid::CodroidClient robot;
```

~CodroidClient()

Destructor. Automatically cleans up resources if `Disconnect()` was not called.

Connection Management

Connect

```
bool Connect(const std::string& ip, int port = 9001);
```

Establish TCP connection only, no mode switch, no power on.

Parameter	Type	Description
<code>ip</code>	<code>string</code>	Controller IP address
<code>port</code>	<code>int</code>	TCP port, default 9001

Returns: `true` on success

ConnectRemoteAndSwitchOn

```
bool ConnectRemoteAndSwitchOn(const std::string& ip, int port = 9001, std::string local_ip = {});
```

Connect and perform remote mode switch + power on. When `local_ip` is non-empty, it's used for `StartCriDataPush` UDP binding.

Parameter	Type	Description
<code>ip</code>	<code>string</code>	Controller IP address
<code>port</code>	<code>int</code>	TCP port, default 9001
<code>local_ip</code>	<code>string</code>	Local IP for CRI UDP binding

Returns: `true` on success

Recommended: Use this method for most scenarios.

Disconnect

```
void Disconnect();
```

Disconnect TCP, stop CRI threads and cache.

Must call: Call before program exit to ensure resource cleanup.

NextRequestId

```
int NextRequestId();
```

Generate next monotonically increasing request ID. Avoid duplicates in multi-threaded commands.

Mode Control

SwitchOn / SwitchOff

```
CommandResult SwitchOn(int id = 1);
CommandResult SwitchOff(int id = 1);
```

Robot power on / off.

ToManual / ToAuto / ToRemote

```
CommandResult ToManual(int id = 1);
CommandResult ToAuto(int id = 1);
CommandResult ToRemote(int id = 1);
```

Switch to manual/auto/remote mode.

ClearSystemError

```
CommandResult ClearSystemError(int id = 1);
```

Clear system error.

EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
CommandResult EnterManualModeViaAuto(int id = 1);  
CommandResult EnterRemoteModeViaAuto(int id = 1);
```

Switch to auto first, then to manual/remote.

ToSimulation / ToActual

```
CommandResult ToSimulation(int id = 1);  
CommandResult ToActual(int id = 1);
```

Enter simulation/actual mode.

StartDrag / StopDrag

```
CommandResult StartDrag(int id = 1);  
CommandResult StopDrag(int id = 1);
```

Enter/exit drag teaching mode.

IO Operations

GetDi / GetDo / GetAi / GetAo

```
int GetDi(int port, int id = 1);  
int GetDo(int port, int id = 1);  
double GetAi(int port, int id = 1);  
double GetAo(int port, int id = 1);
```

Read digital/analog input/output.

SetDo / SetAo

```
CommandResult SetDo(int port, int value, int id = 1);  
CommandResult SetAo(int port, double value, int id = 1);
```

Set digital/analog output.

Register Operations

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);  
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id  
= 1);
```

Read register values.

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

Set register value.

Motion Control

MovJ / MovL / MovC / MovCircle

```
CommandResult MovJ(const ClientJointPoint& target, double speed, double acceleration, int id  
= 1);  
CommandResult MovJ(const ClientCartesianPoint& target, double speed, double acceleration,  
int id = 1);  
CommandResult MovL(const ClientCartesianPoint& target, double speed, double acceleration,  
...);  
CommandResult MovC(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,  
...);  
CommandResult MovCircle(const ClientCartesianPoint& middle, const ClientCartesianPoint&  
target, int circle_num, ...);
```

Motion commands.

Move / MovePath

```
CommandResult Move(const std::vector<ClientMoveInstruction>& path, int id = 1);  
CommandResult MovePath(const std::vector<ClientMoveInstruction>& path, int id = 1);
```

Multi-segment path execution.

Blocking Motion (Sync)

```
bool MoveSync(const std::vector<ClientMoveInstruction>& path, const MotionWaitOptions& wait  
= {});  
bool MovJSync(const ClientJointPoint& target, double speed, double acc, const  
MotionWaitOptions& wait = {});  
bool MovLSync(const ClientCartesianPoint& target, double speed, double acc, const  
MotionWaitOptions& wait = {});  
bool MovCSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target, ...);  
bool MovCircleSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,  
int circle_num, ...);
```

Blocking motion, waits for CRI confirmation of arrival.

Prerequisite: Before calling `*Sync` methods, ensure CRI data push has started (call `StartCriDataPush` and wait for first frame).

PauseRobotMotion / ResumeRobotMotion / StopRobotMove

```
CommandResult PauseRobotMotion(int id = 1);  
CommandResult ResumeRobotMotion(int id = 1);  
CommandResult StopRobotMove(int id = 1);
```

Pause, resume, stop motion.

MoveTo (Planned Motion)

MoveTo

```
CommandResult MoveTo(const MoveToParams& params, int id = 1);
```

MoveTo preset/planned motion.

MoveToHeartbeat

```
CommandResult MoveToHeartbeat(int id = 1);
```

MoveTo heartbeat. Call every ≥ 500 ms when using `MoveToType::Joint` or `Line`.

StopMoveTo

```
CommandResult StopMoveTo(int id = 1);
```

Stop MoveTo motion.

Jog

Jog

```
CommandResult Jog(const JogParams& params, int id = 1);
```

Jog motion.

StopJog / JogHeartbeat

```
CommandResult StopJog(int id = 1);  
CommandResult JogHeartbeat(int id = 1);
```

Stop jog / jog heartbeat.

CRI Real-Time Control

StartCriDataPush / StopCriDataPush

```
CommandResult StartCriDataPush(const std::string& udpIp, int udpPort, int id = 1);  
CommandResult StopCriDataPush(int id = 1);
```

Start/stop CRI data push.

StartCriControl / StopCriControl

```
CommandResult StartCriControl(int filterType, int durationMs, int startBuffer, int id = 1);  
CommandResult StopCriControl(int id = 1);
```

Start/stop real-time control session.

GetRobotRealtimeState

```
ClientRealtimeState GetRobotRealtimeState() const;
```

Get thread-safe CRI data snapshot.

SetCriDataReceived

```
void SetCriDataReceived(std::function<void(const ClientRealtimeState&)> cb);
```

Set CRI data callback.

WaitForCriData

```
void WaitForCriData(double timeout_s = 5.0);
```

Block until first CRI data frame arrives.

Kinematics

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);  
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

Forward/inverse kinematics.

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

Cartesian relative pose calculation.

Global Variables

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);  
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);  
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

Global variable operations.

Publish/Subscribe

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

Subscribe to publish topic.

Project/Script

RunScript / EnterRemoteScriptMode / Run / RunByIndex / RunStep

```
CommandResult RunScript(const std::string& mainScript, ...);  
CommandResult EnterRemoteScriptMode(int id = 1);  
CommandResult Run(const std::string& projectId, int id = 1);  
CommandResult RunByIndex(int index, int id = 1);  
CommandResult RunStep(const std::string& projectId, int id = 1);
```

Project/script operations.

PauseProject / ResumeProject / StopProject

```
CommandResult PauseProject(int id = 1);  
CommandResult ResumeProject(int id = 1);  
CommandResult StopProject(int id = 1);
```

Pause, resume, stop project.

Robot Settings

GetRobotParameters

```
ClientRobotParameters GetRobotParameters(int id = 1);
```

Get settings parameters.

SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
CommandResult SetDefaultPayloadId(int payloadId, int id = 1);
CommandResult SetDefaultToolId(int toolId, int id = 1);
CommandResult SetDefaultUserCoordinateId(int coordinateId, int id = 1);
```

Set default payload, tool, user coordinate ID (1~15).

SaveToolFrames / SetToolFrame / SavePayloadFrames / SetPayloadFrame / SetUserCoordinateFrame

```
CommandResult SaveToolFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
CommandResult SetToolFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
CommandResult SavePayloadFrames(const std::vector<ClientRobotPayload>& frames, int id = 1);
CommandResult SetPayloadFrame(int frame_id, const ClientRobotPayload& frame, int id = 1);
CommandResult SetUserCoordinateFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

Save/modify coordinate frames.

SetPayload / SetManualMoveRate / SetAutoMoveRate / SetCollisionSensitivity

```
CommandResult SetPayload(int payloadId, int id = 1);
CommandResult SetManualMoveRate(int pct, int id = 1);
CommandResult SetAutoMoveRate(int pct, int id = 1);
CommandResult SetCollisionSensitivity(int sensitivity, int id = 1);
```

Runtime parameter settings.

Error Handling Settings

SetThrowOnCommandError / ThrowOnCommandError

```
void SetThrowOnCommandError(bool enable);
bool ThrowOnCommandError() const noexcept;
```

Set/get error handling mode.

Motion API Reference

Point Types

JointPoint

Joint space target point (six axes, unit: degrees).

```
struct JointPoint {  
    std::vector<double> jp; // Six joint angles (degrees)  
  
    static JointPoint Degrees(std::vector<double> joints_deg);  
};
```

Usage:

```
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});  
robot.MovJ(joints, 40, 100);
```

CartesianPoint

Cartesian end-effector target point (TCP pose: mm + degrees).

```
struct CartesianPoint {  
    std::vector<double> cp; // [x, y, z, rx, ry, rz]  
    std::vector<double> rj; // Inverse kinematics reference joints (degrees)  
  
    static CartesianPoint MmDeg(std::vector<double> pose_mm_deg);  
    static CartesianPoint MmDegWithRef(std::vector<double> pose_mm_deg, std::vector<double>  
ref_joints_deg);  
};
```

Usage:

```
// Without reference joints  
auto pose1 = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});  
  
// With reference joints (recommended)  
auto state = robot.GetRobotRealtimeState();  
auto pose2 = Codroid::CartesianPoint::MmDegWithRef(  
    {400, 0, 300, 180, 0, 0},  
    state.joint_position  
);
```

MovePoint

Single target point in path segment (protocol layer).

```
struct MovePoint {
    std::vector<double> jp;
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;

    static MovePoint Joint(JointPoint joint);
    static MovePoint Cartesian(CartesianPoint cart);
};
```

Path Instructions

ClientMoveInstruction / MoveInstruction

Single segment in a path.

```
struct ClientMoveInstruction {
    ClientMoveType type = ClientMoveType::MovJ;
    double speed = 60.0;
    double acceleration = 150.0;
    double blend = -1.0;
    double relative_blend = -1.0;
    int circle_num = 1;
    ClientMovePoint target;
    ClientMovePoint middle;
    std::vector<double> coor;
    std::vector<double> tool;

    static ClientMoveInstruction MovJ(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovJ(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovC(ClientCartesianPoint middle, ClientCartesianPoint
target, ...);
    static ClientMoveInstruction MovCircle(ClientCartesianPoint middle, ClientCartesianPoint
target, int circle_num, ...);
};
```

Usage:

```

auto state = robot.GetRobotRealtimeState();

std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
state.joint_position),
        150, 500)
};
robot.Move(path);

```

Motion Type Enums

ClientMoveType / MoveType

```

enum class ClientMoveType {
    MovJ,          // Joint motion
    MovL,          // Linear (Cartesian)
    MovC,          // Circular (via midpoint)
    MovCircle      // Full circle / multi-revolution
};

```

Blocking Motion Parameters

MotionWaitOptions

```

struct MotionWaitOptions {
    double timeout_s = 60.0;           // Overall timeout (seconds)
    double poll_interval_s = 0.05;    // CRI poll interval (seconds)
    double cri_stale_timeout_s = 0.5;  // CRI data stale timeout (seconds)
    int settled_samples = 3;           // InMotion=false consecutive stable
samples
    double joint_tolerance_deg = 0.2;  // Joint target tolerance (degrees)
    double cartesian_position_tolerance_mm = 1.0; // Cartesian position tolerance (mm)
    double cartesian_orientation_tolerance_deg = 1.0; // Cartesian orientation tolerance
(degrees)
};

```

MoveTo Parameters

MoveToType

```
enum class MoveToType : int {
    Stop = -1,          // Stop MoveTo
    Home = 0,           // Go Home
    Safe = 1,           // Go to safe position
    Candle = 2,         // Go to candle position
    Packing = 3,        // Go to packing position
    Joint = 4,          // Joint planning to target
    Line = 5,           // Linear planning to target
    ResumePoint = 6
};
```

MoveToParams

```
struct MoveToParams {
    MoveToType type = MoveToType::Home;
    MoveToTarget target;
};
```

Jog Parameters

JogMode

```
enum class JogMode {
    Joint = 1,  // Joint jog
    Line = 2    // Linear jog
};
```

JogParams

```
struct JogParams {
    JogMode mode = JogMode::Line;
    double speed = 0.0;          // -1~1 ratio
    int index = 1;               // Axis number or direction
    CoordType coordType = CoordType::User;
    int coordId = 1;
};
```

Data Types and Enums

Communication

CommandResult

```
struct CommandResult {
    int id = 0;
    std::string ty;
    std::string error_msg;
    std::string raw_json;

    bool Ok() const noexcept;
};
```

Real-Time Data

ClientRealtimeState

```
struct ClientRealtimeState {
    int64_t timestamp_ms = 0;
    bool data_valid = false;
    uint16_t status1_raw = 0;
    uint16_t status2_raw = 0;

    std::vector<double> joint_position;           // Joint position, degrees
    std::vector<double> joint_velocity;           // Joint velocity, deg/s
    std::vector<double> tcp_pose;                 // [x,y,z,rx,ry,rz], mm + degrees
    std::vector<double> tcp_velocity;             // Linear mm/s, angular °/s
    double tcp_linear_velocity_mm_s = 0.0;
    std::vector<double> joint_output_torque;
    std::vector<double> joint_external_force;
    std::vector<double> external_axis_position;

    bool project_running = false;
    bool project_stopped = false;
    bool project_paused = false;
    bool enabling = false;
    bool not_enabled = false;
    bool manual_mode = false;
    bool dragging = false;
    bool in_motion = false;
    bool collision_stopped = false;
    bool in_safety_position = false;
    bool has_alarm = false;
    bool simulation_mode = false;
    bool emergency_stop_pressed = false;
    bool rescue_mode = false;
    bool auto_mode = false;
```



```
bool remote_mode = false;
bool realtime_control_mode = false;
uint8_t cri_error_code = 0;
};
```

IO Related

ClientIoInfo / ClientRegisterInfo

```
struct ClientIoInfo {
    std::string type;
    int port = 0;
    double value = 0.0;
};

struct ClientRegisterInfo {
    int address = 0;
    double value = 0.0;
};
```

Robot Settings

ClientRobotFrame / ClientRobotPayload

```
struct ClientRobotFrame {
    int id = 0;
    double x = 0.0, y = 0.0, z = 0.0;
    double a = 0.0, b = 0.0, c = 0.0;
};

struct ClientRobotPayload {
    int id = 0;
    double m = 0.0;
    double mx = 0.0, my = 0.0, mz = 0.0;
};
```

ClientRobotParameters

```
struct ClientRobotParameters {
    bool valid = false;
    int default_tool_id = 0;
    int default_payload_id = 0;
    int default_coordinate_id = 0;
    double max_payload = 0.0;
    std::vector<ClientRobotFrame> tool;
    std::vector<ClientRobotPayload> payload;
    std::vector<ClientRobotFrame> coordinate;
};
```

Publish/Subscribe

ClientPublishNotification

```
struct ClientPublishNotification {
    std::string ty;
    std::string db_json;
    std::string raw_json;
};
```

ClientPublishSubscription

```
class ClientPublishSubscription {
public:
    void Dispose();
    bool IsValid() const noexcept;
};
```

Global Variables

Variable

```
struct Variable {
    std::string val;
    std::string nm;

    template<typename T>
    Variable(const T& value, const std::string& note = "");
};
```

Kinematics

FKParams / IKParams

```
struct FKParams {
    std::vector<double> jp;
    std::vector<double> coor, tool, ep;
};

struct IKParams {
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;
};
```

RelativePoseParams

```
struct RelativePoseParams {
    std::vector<double> pos;
    std::vector<double> offset;
    CoordType coordType = CoordType::Tool;
    std::vector<double> posCoord;
    std::vector<double> coord;
};
```

Exceptions

CodroidException / CodroidCommandException

```
class CodroidException : public std::runtime_error {
public:
    explicit CodroidException(const std::string& message);
};

class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;
    const std::string& command_ty() const noexcept;
    const std::string& controller_error() const noexcept;
    const std::string& raw_response_json() const noexcept;
};
```

Firmware Version Constant

```
inline constexpr const char* MinControllerFirmware = "2.3.3.43";
```


SendCommand

```
void SendCommand(const std::array<double, 6>& position6, TrajectorySpace space);
```

SendTrajectory

```
void SendTrajectory(const std::vector<TrajectoryPoint>& trajectory, TrajectorySpace space,
int period_ms,
                const std::atomic<bool>* cancel = nullptr);
```

Close / IsOpen

```
void Close() noexcept;
bool IsOpen() const noexcept;
```

TrajectoryGenerator

Offline trajectory generation.

```
#include "Codroid/trajectory_generator.hpp"
```

Generate

```
static std::vector<TrajectoryPoint> Generate(const std::array<double, 6>& start,
                const std::array<double, 6>& target,
                const TrajectoryRequest& request);
```

GenerateMultiSegment

```
static std::vector<TrajectoryPoint> GenerateMultiSegment(const
std::vector<std::array<double, 6>>& waypoints,
                const TrajectoryRequest& request);
```

TrajectoryRequest

```
struct TrajectoryRequest {
    TrajectorySpace space = TrajectorySpace::Joint;
    TrajectoryProfile profile = TrajectoryProfile::Trapezoidal;
    double duration_s = 1.0;
    double frequency_hz = 250.0;
    double max_velocity = 0.0;
    double max_acceleration = 0.0;
    double max_jerk = 0.0;
};
```

TrajectorySpace / TrajectoryProfile

```
enum class TrajectorySpace { Joint, Cartesian };
enum class TrajectoryProfile { Cubic, Trapezoidal };
```

Complete Example

```
#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"

// 1. Connect
Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");

// 2. Start CRI data push
robot.StartCriDataPush("192.168.8.150", 9030);
robot.WaitForCriData(5.0);

// 3. Start real-time control
robot.StartCriControl(1, 4, 5);

// 4. Wait for real-time control mode
while (!robot.GetRobotRealtimeState().realtime_control_mode) {
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// 5. Generate trajectory
auto state = robot.GetRobotRealtimeState();
std::array<double, 6> start;
for (int i = 0; i < 6; i++) start[i] = state.joint_position[i];

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(
    start, {10, 0, 90, 0, 90, 0}, request);

// 6. Send trajectory
Codroid::CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, Codroid::TrajectorySpace::Joint, 4);

// 7. Cleanup
robot.StopCriControl();
robot.StopCriDataPush();
robot.Disconnect();
```

IO and Register API Reference

Digital IO

GetDi / GetDo

```
int GetDi(int port, int id = 1);
int GetDo(int port, int id = 1);
```

SetDo

```
CommandResult SetDo(int port, int value, int id = 1);
```

Analog IO

GetAi / GetAo

```
double GetAi(int port, int id = 1);
double GetAo(int port, int id = 1);
```

SetAo

```
CommandResult SetAo(int port, double value, int id = 1);
```

Registers

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id = 1);
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

Complete Example

```
#include "Codroid/client.hpp"

Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
```

```
// IO operations
int di0 = robot.GetDi(0);
robot.SetDo(10, di0);

double ai0 = robot.GetAi(0);
robot.SetAo(0, 3.3);

// Register operations
double value = robot.GetRegisterValue(49100);
robot.SetRegisterValue(49100, value + 1);

// Batch read
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);

robot.Disconnect();
```


Utilities API Reference

Publish/Subscribe

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

Available Topics

Topic	Description
publish/ProjectState	Project running state
publish/VarUpdate	Variable update
publish/RobotStatus	Robot status
publish/RobotPosture	Robot posture
publish/RobotCoordinate	Coordinate system
publish/Log	Log
publish/Error	Error

PublishTopics Constants

```
struct PublishTopics {  
    static constexpr const char* ProjectState = "publish/ProjectState";  
    static constexpr const char* VarUpdate = "publish/VarUpdate";  
    static constexpr const char* RobotStatus = "publish/RobotStatus";  
    static constexpr const char* RobotPosture = "publish/RobotPosture";  
    static constexpr const char* RobotCoordinate = "publish/RobotCoordinate";  
    static constexpr const char* Log = "publish/Log";  
    static constexpr const char* Error = "publish/Error";  
};
```

Global Variables

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

Kinematics

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

Console UTF-8

ConsoleUtf8

```
#include "Codroid/console_utf8.hpp"

// Windows console UTF-8 support
#ifdef _WIN32
Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif
```

Complete Example

```
#include "Codroid/client.hpp"

// Publish/Subscribe
auto sub = robot.SubscribePublishTopic(
    Codroid::PublishTopics::RobotStatus,
    [](const Codroid::ClientPublishNotification& n) {
        std::cout << "Status: " << n.db_json << std::endl;
    }
);

// Global variables
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "Counter")},
```

```
    {"name", Codroid::Variable(std::string("test"), "Name")}
});

// Kinematics
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});
auto tcp = robot.ForwardKinematics(fk);

Codroid::IKParams ik({400, 0, 300, 180, 0, 0});
ik.rj = {0, 0, 90, 0, 90, 0};
auto joints = robot.InverseKinematics(ik);
```